

2-Bit Branch Predictor Implementation and Performance

To improve control hazard handling, a **2-bit saturating branch predictor** was implemented in the fetch stage. The predictor uses a 2-bit state machine with four states: *strongly not taken* (00), *weakly not taken* (01), *weakly taken* (10), and *strongly taken* (11). The most significant bit determines the prediction (0 = not taken, 1 = taken).

During execution, the predictor is updated in the execute stage based on the actual branch outcome. If a branch is taken, the state increments toward strongly taken; if not taken, it decrements toward strongly not taken. Updates are only applied to branch instructions (excluding JALR and JAL), ensuring accurate training.

In the fetch stage, when a branch or JAL instruction is encountered, the predictor determines whether to speculatively fetch the branch target or the next sequential instruction (PC+4). If the prediction is incorrect, the pipeline is flushed and the correct PC is restored using the resolved target or saved PC+4.

Compared to the baseline design (static predict-not-taken), this dynamic predictor reduces misprediction penalties, especially in loop-heavy code such as sorting. As a result, the CPI improved from:

- **6.5833 → 6.25**

This improvement reflects fewer pipeline flushes and better handling of frequently taken branches, demonstrating the effectiveness of even a simple dynamic prediction scheme.

Unfortunately JALR is always predicted not to be taken, even though I know it is always taken, but considering that it needs to read the register and do the operation, I reserve it outside.